

paper-agent 最終仕様書

論文執筆エージェント 構成仕様書

Version: 1.2 | Date: 2026-06-02 | Target field (Phase 1): 理学・工学 | Repository:
<https://github.com/dan23514/paper-agent>

1. 概要

1.1 目的

ユーザーから提供される研究要素（テーマ・手法・データ・結果・解釈）を入力として、Q1/Q2 ジャーナル掲載論文のみを引用した日本語の学術論文ドラフトを生成し、ファクトチェック・マルチAIレビューを経て Word (.docx) 形式で出力する半自動エージェントを構築する。

1.2 設計原則

#	原則	内容
1	Claude Code 単一環境完結	追加フレームワーク（LangGraph 等）を導入せず、Claude Code のスラッシュコマンド・Subagent・Skill・MCP で構成する
2	Human-in-the-loop によるコスト圧縮	ChatGPT・Gemini のレビューはユーザーが Web 版を手動で呼び出すリレー方式とし、API 従量課金を回避する。Claude レビューは Claude Code 内で自動実行する
3	引用ファクトチェックの独立化	汎用 LLM の引用捏造リスクを排除するため、ファクトチェックは NotebookLM に分離する
4	チェックポイント駆動	全自動ではなく、明示的なチェックポイントごとに成果物を出力し、ユーザーが介入・承認してから次工程に進む
5	引用捏造防止	<code>verify_refs.py</code> が本文中の全 <code>[REF: Rx]</code> を候補リスト ID と照合。無効 ID は即時リジェクト・再生成
6	DOI 検証優先	文献は必ず CrossRef API で DOI・タイトル・年を实在確認してから BibTeX を採用する

1.3 想定コスト

- Claude Code Pro または Max サブスクリプション

- OpenAlex API：完全無料（登録不要）
- CrossRef API：完全無料
- Exa MCP：補完用途のみのため無料枠内に収まる想定
- NotebookLM / ChatGPT (web) / Gemini (web)：いずれも無料枠を活用

2. システム構成

2.1 コンポーネント一覧

ID	コンポーネント	種別	役割
C1	Input Loader	Slash Cmd <code>/load</code>	入力ファイルから研究要素を抽出し cp0_input.yaml を生成
C2	Literature Searcher	Subagent + OpenAlex API + Exa MCP	Q1/Q2 限定で関連文献を検索
C3	BibTeX Fetcher	Script <code>bib_fetcher.py</code> + CrossRef API	DOI から BibTeX/CSL-JSON を取得
C4	SJR Filter	Skill <code>sjr-filter</code>	Scimago Journal Rank データで Q1/Q2 判定
C5	Outline Generator	Subagent <code>outliner</code>	IMRaD 構成のアウトライン生成
C6	Writer	Subagent <code>writer</code>	セクション別に本文・Abstract・Keywords を執筆
C7	Citation Manager	Skill <code>citation</code>	引用整形（IEEE/ACS/AIP/Nature/APA 等）
C8	Factcheck Packet Builder	Slash Cmd <code>/factcheck-packet</code>	NotebookLM 用パッケージ出力
C9	Review Packet Builder	Slash Cmd <code>/review-packet</code>	外部 AI レビュー用パッケージ出力 + Claude 自動レビュー起動
C10	Integrator	Subagent <code>integrator</code>	レビュー・Factcheck 結果を本文に統合
C11	DOCX Exporter	Skill <code>docx-export</code>	python-docx で Word 出力

2.2 MCP 構成

Server	用途	コスト
Exa	補完文献検索（OpenAlex でカバレッジ不足の場合）	無料枠あり（補完用途のみで枠内に収まる想定）
Google Drive（任意）	出力ファイルの共有	無料

3. ワークフロー（CP0～CP7）



3.1 各 CP の入出力

CP	入力	主な出力	ユーザー確認
CP0	input.md / input.yaml	cp0_input.yaml、 progress.yaml、 config.yaml	欠損フィールドの警告確認
CP1	cp0_input.yaml、 config.yaml	cp1_references.md、 cp1_references.bib	引用候補リストの編集・承認
CP2	cp0_input.yaml、 cp1_references.md	cp2_outline.md	アウトラインの編集・承認
CP3	cp2_outline.md、 cp1_references.md	cp3_draft_v1.md	ドラフトの確認
CP4	cp3_draft_v1.md	cp4_review_packet/ (results/claude.md 自動生成)	ChatGPT・Gemini への投稿と結果保存
CP5	cp3_draft_v1.md、 cp1_references.json	cp5_factcheck_packet/ (claims.md、 pdfs/)	NotebookLM での検証と results.md 記入
CP6	レビュー結果、 Factcheck 結果	cp6_draft_v2.md、 cp6_revision_report.md	diff の承認/却下
CP7	cp6_draft_v2.md、 config.yaml	cp7_final.md、 output/*.docx	最終確認

4. 文献検索仕様

4.1 OpenAlex API（主検索）

- Endpoint: `https://api.openalex.org/works`
- Polite pool: クエリに `mailto=<user_email>` を付与してレート制限を緩和（10 req/sec）
- フィールド絞り込み: `select=` で取得フィールドを限定し帯域を節約
- 収録論文数: 2.5 億件以上（arXiv・PubMed Central・DOAJ 等を含む）

4.2 Q1/Q2 フィルタリング

Scimago Journal Rank (SJR) の年次 CSV（`data/sjr_YYYY.csv`）を使用。

優先順位	マッチング方法	閾値
1位	ISSN（完全一致）	—
2位	誌名の正規化 + Levenshtein 類似度	≥ 0.85
特例	boost_journals 指定誌は SJR バイパス可（bypass_sjr_for_targets: true）	—

4.3 ターゲットジャーナル・ブースト

`boost_journals` を指定すると、指定誌の論文に係数（`target_boost`、デフォルト 1.5）を乗算して優先度を上げる。上限制御（`max_target_citation_ratio`、デフォルト 0.4）でエコーチェンバー化を防ぐ。

5. Factcheck 仕様

5.1 Factcheck Packet 構成

factcheck_packet/		
├ claims.md	本文中の全引用主張（主張 + 引用ID + DOI のテーブル）	
├ pdfs/	自動取得した OA-PDF（PDF マジックバイト検証済み）	
├ missing_pdfs.md	ユーザーが手動入手すべき PDF リスト	
├ notebooklm_prompts.md	NotebookLM への検証プロンプト集	
├ instructions.md	NotebookLM 操作マニュアル	
└ results.md	結果記入テンプレート（空）	

5.2 NotebookLM の判定基準

Verdict	意味	/integrate-factcheck での処理
SUPPORTED	主張は論文に明確に支持されている	変更なし
PARTIAL	部分的に支持されている	表現を緩和する修正案を提示
NOT_SUPPORTED	論文に該当箇所がない	引用削除・表現変更・代替文献提案

6. マルチ AI レビュー仕様

観点	AI	方式
論理構成・文体	Claude (reviewer Subagent)	自動実行
学術的正確性・先行研究の網羅性	ChatGPT (with search)	Human-in-the-loop
統計・データ解釈の妥当性	Gemini	Human-in-the-loop

6.1 統合ロジック

- `[CRITICAL]` タグ付きの指摘 → 必ず対応
- 2つ以上の AI が同じ箇所を指摘 → 高優先度として必ず対応
- 改善提案は diff 形式でユーザーに提示し承認制

7. 出力仕様

7.1 生成ライブラリ

`python-docx` (テンプレートファイル対応)

7.2 設定可能パラメータ

パラメータ	デフォルト	選択肢
font_family_jp	游明朝	MS明朝、ヒラギノ明朝 等
font_family_en	Times New Roman	Cambria、Arial 等
font_size_body	10.5pt	9～12pt
line_spacing	1.5	1.0 / 1.15 / 1.5 / 2.0
paper_size	A4	A4 / Letter
citation_style	IEEE	IEEE / ACS / AIP / Nature / Vancouver / APA
template	(なし)	templates/*.docx へのパス (学会指定テンプレート)

8. エラー回復・状態管理

各プロジェクトの `checkpoints/progress.yaml` で進捗を管理する。

- 各スラッシュコマンドは起動時に `progress.yaml` を確認し、未完了の箇所から再開する
- セクション・ステップ完了のたびに即時更新（クラッシュ時の工程損失を最小化）
- エラー発生時は `error_log` に追記してから処理を停止
- ユーザーは `progress.yaml` の `current_cp` を手動編集して特定の CP に巻き戻せる

9. 将来拡張計画

Phase	内容
2	分野追加（医学・生命科学 / 社会科学・人文科学）
3	Local RAG による補助 Factcheck（NotebookLM の上限・手動操作を削減）
4	API 経由のフル自動化オプション（ChatGPT / Gemini API モード）
5	図表生成（matplotlib / plotly での自動図）
6	リバイズ対応モード（査読コメント → 反論・修正案生成）
7	Cover Letter / 投稿先推薦機能